University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Operating System

Lab No 5

Process Creation using Fork ()

Faculty of Information Technology

UCP Lahore Pakistan

Instructor Name Here, Department of Software Engineering

University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

Process Creation in Linux Enviornment

Topic to be covered

- *Process Creation*
  - fork ()
  - getpid()
  - getppid()
- *Process Completion*
  - wait (int *)
  - exit (int)
- *Orphan Process*
- Zombie Process
- Multiple forks in a program.
- Example Programs for proof of concept

Objectives

- *Understanding the concept of creating new processes in Linux.*
- *Understanding the process id and parent process id using getpid() and getpid()*
- *Parent waiting for its child to terminate using wait() system call.*
- Concept of orphan and zombie Processes.
- Understanding the use of multiple forks in a program using tree diagram.

Prerequisite:

- Visual Studio Code
- GCC compiler
- Basic c Programing
- Use of man Page

University of Central Punjab

FACULTY OF INFORMATION TECHNOLOGY

Lab Questions - Process Creation Using fork()

Question 1: Parallel Task Execution Using fork()

Concept: Process creation and parent-child execution.

A file-processing application needs to perform two independent tasks: one process displays the numbers from 1 to 5, while another displays the letters A to E.

Write a C program that creates a child process using fork().

Requirements:

1. The child process must display the letters A to E.
2. The parent process must display the numbers 1 to 5.
3. Both processes must display their PIDs.
4. Handle the failure of fork().
5. Use wait() so the parent terminates only after its child has completed.

Expected behavior: Both sequences should be displayed, and the parent should finish after the child. The order in which the two sequences appear may vary.

Sample Output:

```
Parent Process PID: 2450
Child Process PID: 2451
Child: A
Parent: 1
Child: B
Parent: 2
Child: C
Parent: 3
Child: D
Parent: 4
Child: E
Parent: 5
Parent: Child process completed.
```

Note: PIDs and the order of parent/child output may vary.

University of Central Punjab

Question 2: Execute Another Program Using exec()

Concept: Process creation and program replacement.

A system-monitoring application must create a child process that executes the Linux ls -l command, while the parent waits for the command to finish.

Write a C program that:

1. Creates a child using fork().
2. Uses an appropriate exec() family function in the child to execute ls -l.
3. Displays an error if the execution fails.
4. Uses wait() in the parent to wait for the child.
5. Displays Command execution completed after the child terminates.

Expected behavior: The directory listing appears, followed by the parent's completion message. The child should use perror() if exec() fails, because a successful exec() replaces the current program and does not return.

Sample Output:

```
Parent Process: Waiting for child...
Child Process: Executing ls -l
total 24
-rwxr-xr-x 1 user user 16000 Oct 24 program
-rw-r--r-- 1 user user   800 Oct 24 program.c
Parent Process: Child process completed.
```

Note: The files, permissions, sizes, dates, and PIDs depend on the system and current directory.

Question 3: Child Completion and Exit Status

Concept: fork(), wait(), and exit().

A parent process launches a child to calculate the sum of integers from 1 to 10. The child must display the calculated sum and terminate with exit status 5. The parent must wait for the child and display its exit status.

Write a C program that:

1. Creates a child using fork().
2. Calculates and prints the sum in the child.
3. Terminates the child using exit(5).
4. Retrieves the child's termination status using wait() and the appropriate macros.
5. Displays the exit status only if the child exited normally.

Sample Output:

```
Child Process: Calculating sum from 1 to 10
Child Process: Sum = 55
Parent Process: Child process completed.
Parent Process: Child exit status = 5
```

Note: The exit status is 5 because the child is instructed to terminate using exit(5).

Question 4: Demonstrate an Orphan Process

Concept: Orphan process and getppid().

A parent process starts a child that must continue running after the parent terminates.

Write a C program that:

1. Creates a child using fork().
2. The parent displays its PID and the child's PID, then terminates without waiting.
3. The child sleeps for three seconds.
4. After the delay, the child displays its own PID and current PPID.
5. Explain why the PPID may differ from the original parent's PID.

Expected behavior: The child continues after the original parent exits. Its PPID changes when it is adopted by an appropriate system process. The exact PPID depends on the Linux environment.

Sample Output:

```
Parent Process PID: 3100
Parent Process: Terminating without waiting.
Child Process: Sleeping for 3 seconds...
Child Process PID: 3101
Child Process PPID after sleep: 1
Child Process: I am now an orphan process.
```

Note: The new parent PID may differ. Depending on the Linux environment, an orphan may be adopted by a process other than PID 1.

Question 5: Identify and Resolve a Zombie Process

Concept: Zombie process and wait().

A monitoring application creates a child that terminates immediately, but the parent remains alive without collecting the child's exit status.

University of Central Punjab

Write a C program that:

1. Creates a child using fork().
2. The child prints a message and terminates with exit(0).
3. The parent sleeps for 20 seconds without calling wait().
4. During the delay, inspect the process state from another terminal using ps.
5. Modify the program so the parent calls wait() and explain how the child process state changes.

Expected observation: While the parent remains alive and has not collected the child's status, the child may appear in the process table with state Z. Calling wait() reaps the terminated child.

Sample Output:

```
Parent Process PID: 4200
Child Process PID: 4201
Child Process: Exiting now.
Parent Process: Sleeping for 20 seconds without calling wait().
```

During the parent's sleep, run this command in another terminal:

```
ps -o pid,ppid,stat,cmd -p 4201
```

Sample output from the `ps` command:

```
  PID    PPID STAT CMD
 4201    4200 Z    [zombie_demo] <defunct>
```

Note: Replace `4201` with the actual child PID. The zombie may not appear if the parent has already terminated or reaped the child. After modifying the program to call `wait()`, the child should be reaped instead of remaining a zombie.

Question 6: Analyze fork() && fork()

Concept: Multiple process creation and logical AND.

A developer writes the following expression to create processes:

```
fork() && fork();
printf("Process created\n");
```

Write a complete C program using this expression.

Requirements:

1. Check and explain how the return value of fork() affects logical AND.
2. Count how many processes execute printf(), assuming both fork operations succeed.
3. Display each process's PID.
4. Draw a process tree.
5. Explain why the second fork() is not executed in every process.

Expected result: Process created is printed three times, assuming both calls succeed.

Sample Output:

```
Process created/executing the statement. PID: 5100
Process created/executing the statement. PID: 5101
Process created/executing the statement. PID: 5102
```

Note: Assuming both `fork()` calls succeed, the statement executes in three processes. The PIDs and output order may vary.

Question 7: Analyze fork() || fork()

Concept: Multiple process creation and logical OR.

A developer uses the following expression:

```
fork() || fork();
printf("UCP\n");
```

Write a complete C program using this expression.

Requirements:

1. Explain how short-circuit evaluation affects the second fork().
2. Display the PID of every process reaching the print statement.
3. Count the total processes created, including the original process.
4. Draw a process tree.
5. Compare the result with Question 6.

Expected result: UCP is printed three times, assuming both calls succeed.

Sample Output:

```
Process created/executing the statement. PID: 6100
Process created/executing the statement. PID: 6101
Process created/executing the statement. PID: 6102
```

Note: Assuming both required `fork()` calls succeed, the statement executes in three processes. The PIDs and output order may vary.

Question 8: Combined Process Management Challenge

Concept: fork(), exec(), and wait().

A parent application must launch a child process to display the current date and time using the Linux date command.

Write a C program that:

1. Creates a child process using fork().
2. The child executes date using an exec() family function.
3. If exec() fails, the child displays an error and terminates with a nonzero exit status.
4. The parent uses wait() to collect the child's termination status.
5. The parent displays whether the command completed successfully or failed.

Expected behavior: The date and time appear first, followed by a parent message reporting the child's status.

Sample Output:

```
Parent Process PID: 7200
Parent Process: Waiting for child...
Child Process: Executing date command.
Sat Oct 24 18:30:00 PKT 2026
Parent Process: Child process completed successfully.
```

Note: The date and time will reflect the system's current date and time. PIDs and output details may vary. The success message should only be displayed if the child process finishes successfully.